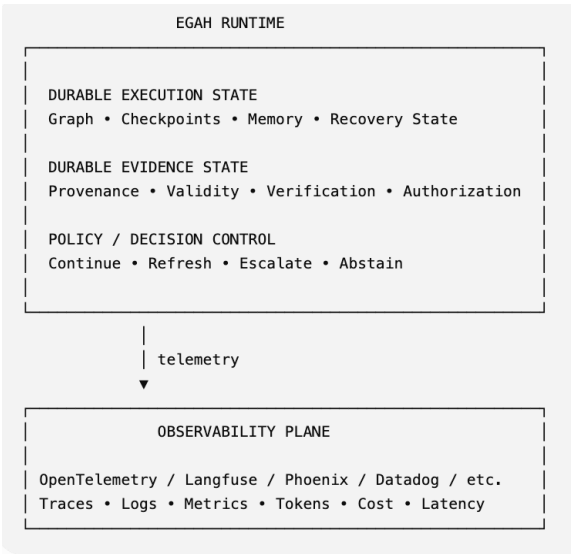
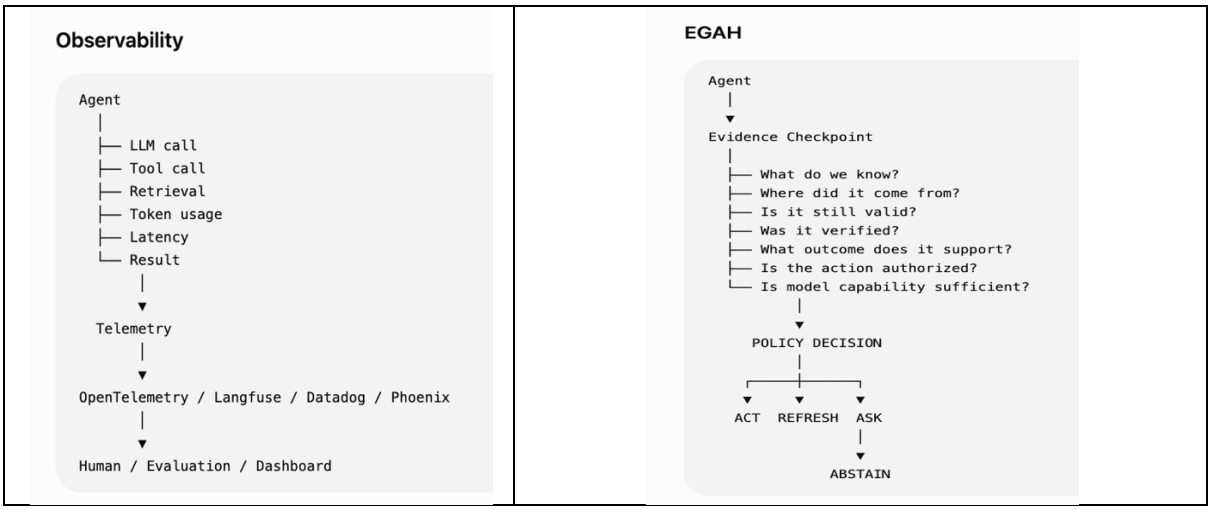
Evidence-Governed Agent Harnesses

What should survive when an agent crashes at step 40 of 60?

An evidence-governed runtime/control layer that consumes observability signals but turns verified, durable evidence into decisions about agent continuation, recovery, escalation and authorization.



While Observability tells us what happened. EGAH asks whether what happened is still sufficient evidence for what the agent is about to do.



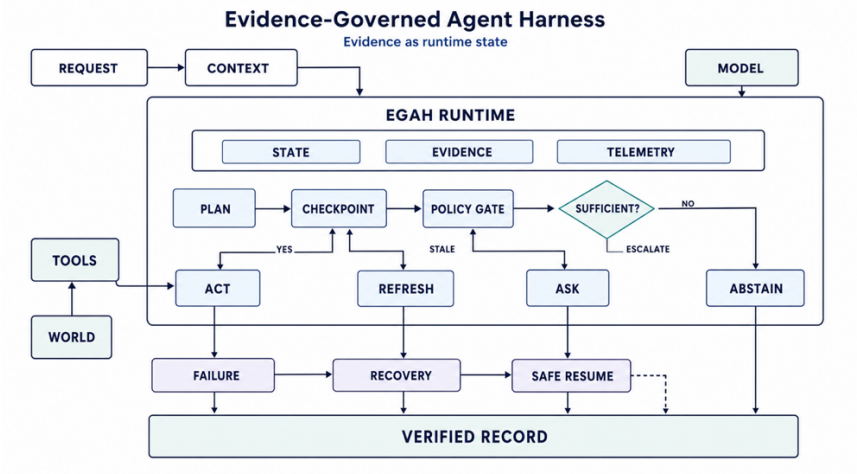
Long-running agents are usually designed to preserve execution state: graph state, checkpoints, memory, tool results, and workflow progress. But in a production agent, state alone is not enough. The agent may resume at step 40 with its execution state intact while the evidence that justified its next action is stale, incomplete, unverified, or no longer applicable. The harness has remembered **where it was**, but not necessarily **what it knew, what it verified, what it accomplished, or what it is still allowed to do**.

This talk explores **Evidence-Governed Agent Harnesses (EGAH):** an approach that treats evidence as first-class runtime state alongside graph state, checkpoints, memory and tool execution. Instead of treating verification and governance as external layers, the harness introduces evidence checkpoints, evidence envelopes, verification history, outcome attribution and capability boundaries directly into the execution graph.

Core shift is from

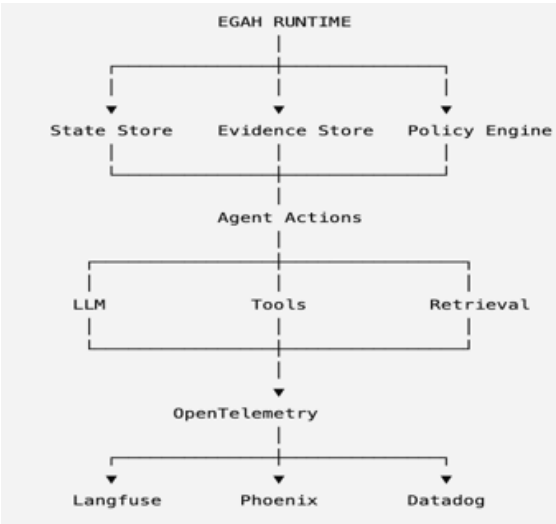
Plan → Tool Call → Result → Next Step to

Plan → Evidence Checkpoint → Tool Call → Result Verification → Evidence Update → Next Step



This becomes particularly important for long-running workflows. If an agent crashes at step 40 of 60, conventional checkpoint recovery can restore execution state, but an evidence-governed harness asks additional questions before continuing:

- Which prerequisites were already verified?
- When were they verified?
- What changed while the workflow was interrupted?
- Which evidence remains valid?
- What must be revalidated before the next consequential action?



The same approach provides a practical way to reason about small-model deployment. Rather than asking whether a tuned 8B model is universally "good enough", the harness can establish **evidence-backed capability boundaries** for specific task classes. Tasks within an established boundary can use a smaller model; boundary-near tasks can trigger verification or human review; tasks beyond the boundary can escalate to a larger model or human intervention.

The talk will walk through harness architecture, evidence-enhanced execution graph, evidence-state model, crash-and-resume scenario, small-model capability boundaries, resource/outcome accounting, and the trade-offs between additional verification and runtime overhead. It will also distinguish clearly between the proposed architecture, implementation experiments, and production validation still required.

The central engineering proposition is simple "**A production agent harness should persist not only where the agent was, but also what the agent knew, what it verified, what it accomplished, and what it is still authorized to do**".

Takeaways

1. **Execution state is not evidence state**. A checkpoint can tell an agent where it was without telling it whether the evidence supporting its next action is still valid.

2. **Make evidence a first-class harness primitive**. Capture provenance, observation time, verification status, applicability, outcome and verification history alongside execution state.

3. **Put verification at consequential boundaries**. The answer is not to validate everything. Use stronger verification where stale or incorrect evidence can materially change the outcome.

4. **Design recovery around semantic continuity**. After a crash, resuming the graph should also consider what evidence needs to be revalidated before execution continues.

5. **Evaluate small models by capability boundaries, not model size alone**. Establish where an 8B model is demonstrably sufficient for a defined task class, where verification is needed, and when escalation is appropriate.

6. **Measure whether the harness earns its complexity**. Checkpoint overhead, recovery correctness, capability-boundary accuracy and outcome quality should determine whether an additional harness mechanism belongs in production.

Audience

Engineers and technical teams building or operating **production AI agents, agent harnesses, execution graphs, tool-using agents and long-running workflows**.

It is particularly relevant to teams moving beyond agent demos and asking what needs to change before an agent can reliably execute **long-running, tool-using and consequential workflows in production**.